3 Launch Template 생성 (User Data 포함)

📍 **위치: AWS Console → EC2 → Launch Template**

User Data (콘솔에 붙여넣기):

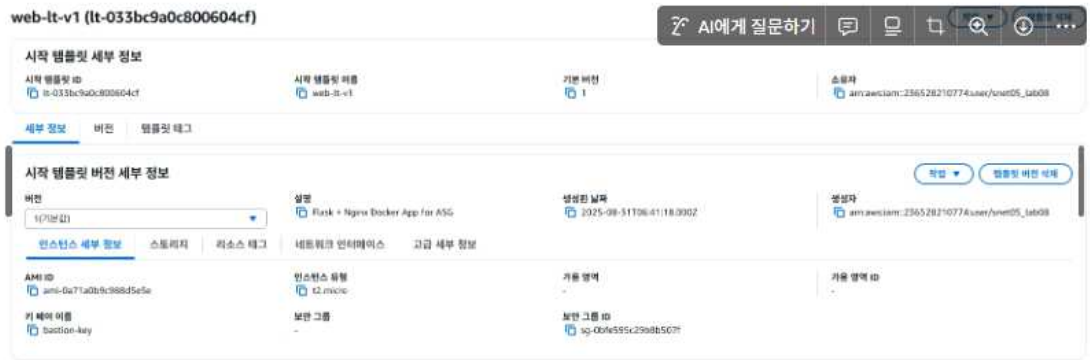
```
#!/bin/bash
apt-get update -y
apt-get install -y docker.io

systemctl start docker
systemctl enable docker

docker pull <DOCKER_ID>/flask-image:1.0
docker run -d -p 80:5000 <DOCKER_ID>/flask-image:1.0
```

👉 **배울 점:**

- Bake 방식과 달리 **User Data**는 인스턴스 부팅 때마다 실행됨.
- → 최신 이미지 바로 반영 가능, 대신 부팅 시간이 조금 늘어남.



4️⃣ Auto Scaling Group 생성

📍 **위치: AWS Console → EC2 → Auto Scaling Groups**

- Launch Template 선택
- Subnet: Web-1a, Web-1c
- Desired = 2, Min = 1, Max = 4
- ALB Target Group 연결, Health Check Path = `/healthz`

👉 **배울 점:**

- 다중 AZ(1a, 1c)에 분산 배치 → 고가용성 확보
- Health Check로 **Unhealthy 인스턴스 자동 제거**

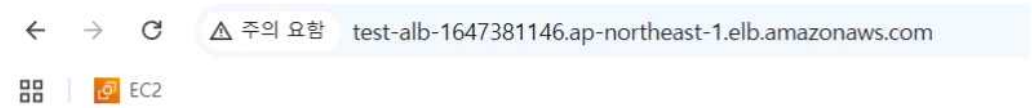
web-asg-v1



5 ALB 동작 확인

📍 위치: 로컬 PC 브라우저

- ALB DNS 접속 → `"Hello from Flask on AWS!"` 출력 확인



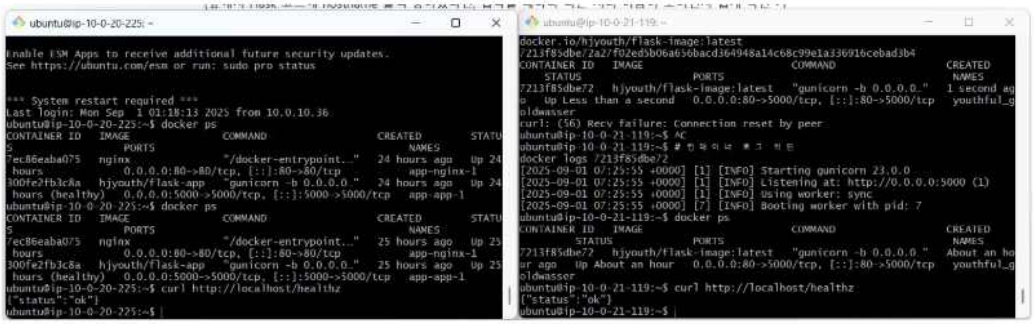
AWS의 Flask에서 안녕하세요!

- AWS Console → Target Group → 인스턴스 상태 = Healthy

👉 **배울 점:**

- ALB는 단순 라운드로빈이 아니라 **헬스체크 기반 무중단 트래픽 관리**

서버 안에서 직접 확인 (Bastion → 각 웹 서버)



이후 진행해야 하는 사항
-Flask 컨테이너 CPU 부하 → Auto Scaling 동작 확인 (현재 단계)
-Nginx 컨테이너 추가 + 리소스 제한 (CPU 20%, 128MB)
→ User Data / Docker Compose 업데이트 후 Instance Refresh
-CloudWatch Agent 설정 → CPU 알람 + SNS 이메일 수신 확인
-GitHub Actions CI/CD 파이프라인 구성
→ 코드 변경 → 자동 배포되는 것 확인